Judgment Geometry, Paper 27:

Galois Cohomology of Abstract Data Types

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

We develop a Galois-cohomological framework for abstract data types (ADTs). An ADT defines a *field extension* $\mathcal{K} \subset \mathcal{L}$ where $\mathcal{K}$ is the interface (public method signatures) and $\mathcal{L}$ is the implementation (private representation type and methods). The *Galois group* $\mathrm{Gal}(\mathcal{L}/\mathcal{K})$ consists of implementation-swapping automorphisms that fix the interface pointwise. The first cohomology $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T}))$ classifies *twisted forms*—implementations that look equivalent locally but differ globally—and precisely characterises representation-independence failures. Our main theorem states: two implementations I_1, I_2 of the same interface are representation-independent if and only if they determine the same class in $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T}))$. We prove an ADT analogue of Hilbert's Theorem 90: $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{GL}_n) = 0$, showing that *linear* data representations are always globally interchangeable. As an application, we certify safe swapping of `HashMap` for `TreeMap` by computing that their cohomology classes coincide. All main results are formalised in Lean 4 with no `sorry`.

Keywords. Galois cohomology, abstract data types, representation independence, Hilbert Theorem 90, twisted forms, Judgment Geometry.

1 Introduction

Modular software systems rely on the principle that an implementation may be replaced by any other implementation of the same interface without observable effect—a property known as *representation independence* [11]. Yet this principle is deceptively subtle: two implementations can satisfy every interface method specification yet fail to be interchangeable when used in combination with other components that exploit hidden implementation details.

A motivating puzzle. Consider a `Map<K,V>` interface exposing `get`, `put`, `delete`, `contains`, and `size`. Both `HashMap` (backed by a hash array) and `TreeMap` (backed by a balanced BST) satisfy this interface. Are they safely interchangeable in all contexts? The answer is *yes for the interface methods*, but potentially *no* if a downstream component inspects iteration order: `TreeMap` guarantees sorted iteration while `HashMap` does not. The extra structure leaks through the interface boundary. This failure is not a bug in either implementation—it is a failure of *global* representation independence even in the presence of *local* interface satisfaction.

Cohomological diagnosis. The Galois cohomology framework introduced in this paper gives a precise algebraic measure of such failures. An ADT defines a tower $\mathcal{K} \hookrightarrow \mathcal{L}$, and the obstructions to global interchangeability are classified by the nonvanishing of $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T}))$. For the `Map` example, the representation type $\mathbf{T}$ is *linear* (an array of entry pairs), so Hilbert 90 forces $H^1 = 0$ and the swap is safe.

Contributions. We make the following contributions.

(1) We define the *field-extension structure* of an ADT: interface as fixed field, implementation as Galois extension, and Galois group as the group of interface-fixing automorphisms (section 2).

(2) We develop *Galois cohomology* for ADTs, define 1-cocycles, coboundaries, and H^1, and introduce *twisted forms* as the geometric objects classified by H^1 (section 4).

(3) We prove the main *representation-independence theorem*: two implementations are representation-independent iff their H^1 classes agree (section 5).

(4) We establish *Hilbert 90 for ADTs*: $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{GL}_n) = 0$ for linear representation types, certifying that all linear ADT implementations are globally interchangeable (section 6).

(5) We provide a *safe-swap algorithm* and apply it to certify `HashMap` $\leftrightarrow$ `TreeMap` swapping (section 7).

(6) We formalise all results in Lean 4 with no `sorry` (section 9).

Relation to previous papers. This paper builds on the Čech cohomology of Paper 3 (*Cohomological Diagnostics*), which classified descent obstructions as elements of H^1 of a semantic site. The present paper specialises that framework to the ADT setting, replacing the sheaf of judgments with the sheaf of implementation automorphisms. The trust algebra of Paper 4 provides the trust annotations attached to swap certificates in section 7. The Galois group construction is the ADT analogue of the arithmetic Galois group studied in Paper 25 (*Arakelov Heights*).

2 ADTs as Field Extensions

We begin by making precise the analogy between ADTs and field extensions that motivates the entire paper.

Definition 2.1 (ADT Interface)**.** An *ADT interface* $\mathcal{K}$ is a finite set of *method signatures*

$$\mathcal{K} = \{m_1 : S_1,\ m_2 : S_2,\ \ldots,\ m_k : S_k\}$$

where each $m_i : S_i$ names a method together with its input/output type S_i. The interface is the *observable part* of the ADT—the collection of operations available to clients.

Definition 2.2 (ADT Implementation)**.** An *implementation* of interface $\mathcal{K}$ is a pair $\mathcal{L} = (\mathbf{T}, \hat{\mathcal{K}})$ where:

- $\mathbf{T}$ is the *representation type*: the private carrier type used to store the ADT's state;
- $\hat{\mathcal{K}} = \{\hat{m}_1, \ldots, \hat{m}_k\}$ is a collection of functions $\hat{m}_i : \mathbf{T} \to S_i$ (or $\mathbf{T} \to \mathbf{T}$ for state-modifying methods) that realise each interface method over $\mathbf{T}$.

The pair $(\mathcal{K}, \mathcal{L})$ mirrors a *field extension* $K \subset L$: the interface $\mathcal{K}$ plays the role of the ground field K (the "fixed" observable data), while the implementation $\mathcal{L}$ extends it with the private representation type $\mathbf{T}$ (the "new elements" adjoined to form L).

Example 2.3 (Stack)**.** The Stack interface is $\mathcal{K}_{\text{stk}} = \{\texttt{push} : \alpha \to (),\ \texttt{pop} : () \to \alpha,\ \texttt{peek} : () \to \alpha,\ \texttt{isEmpty} : () \to \text{bool}\}$. Two implementations are:

- $\mathcal{L}_{\text{arr}}$: representation type $\mathbf{T}_1 = \alpha^* \times \text{int}$ (dynamic array with top pointer);
- $\mathcal{L}_{\text{list}}$: representation type $\mathbf{T}_2 = \mu X.\ (\text{nil} \mid \alpha \times X)$ (algebraic linked list).

Both implement $\mathcal{K}_{\text{stk}}$ but via structurally distinct representation types.

Definition 2.4 (Implementation Morphism)**.** An *implementation morphism* $\phi : \mathcal{L}_1 \to \mathcal{L}_2$ between two implementations of the same interface $\mathcal{K}$ is an isomorphism $\phi : \mathbf{T}_1 \xrightarrow{\sim} \mathbf{T}_2$ such that for every method $m_i \in \mathcal{K}$:

$$\hat{m}_i^{(2)} \circ \phi \ = \ \hat{m}_i^{(1)},$$

i.e. ϕ is a change-of-representation that commutes with every interface method. If such a ϕ exists, we say $\mathcal{L}_1$ and $\mathcal{L}_2$ are *isomorphic implementations*.

The key insight—following the Galois correspondence—is that not every pair of implementations admits such a morphism *globally*, even when they do so *locally* on each method separately. This gap is precisely what Galois cohomology measures.

3 The Galois Group of an ADT

Definition 3.1 ($\mathcal{K}$-Automorphism)**.** A *$\mathcal{K}$-automorphism* of an implementation $\mathcal{L} = (\mathbf{T}, \hat{\mathcal{K}})$ is an automorphism $\sigma \in \text{Aut}(\mathbf{T})$ such that

$$\hat{m}_i \circ \sigma \ = \ \hat{m}_i \qquad \text{for all } m_i \in \mathcal{K}.$$

That is, σ permutes the representation type while leaving every interface method invariant. The set of $\mathcal{K}$-automorphisms forms a group under composition, the *Galois group* of the ADT.

Definition 3.2 (Galois Group)**.** The *Galois group* of an ADT implementation $\mathcal{L}$ of interface $\mathcal{K}$ is

$$G \ = \ \text{Gal}(\mathcal{L}/\mathcal{K}) \ := \ \{\sigma \in \text{Aut}(\mathbf{T}) \ \mid \ \hat{m}_i \circ \sigma = \hat{m}_i \text{ for all } m_i \in \mathcal{K}\}.$$

Example 3.3 (Trivial Galois Group)**.** If every method of $\mathcal{K}$ is *injective* as a function on $\mathbf{T}$ (i.e. the interface completely determines the representation state), then the only $\mathcal{K}$-automorphism is the identity, so $G = \{1\}$. This is the generic situation for "observation-complete" ADTs.

Example 3.4 (Cyclic Galois Group)**.** Consider a `CyclicBuffer<T>` of fixed capacity n with interface $\{\texttt{enqueue}, \texttt{dequeue}, \texttt{size}\}$. The representation $\mathbf{T} = T^n \times \mathbb{Z}/n\mathbb{Z}$ has a natural cyclic symmetry: rotating all n slots simultaneously by one position is an $\mathcal{K}$-automorphism (it does not change what gets enqueued or dequeued, only the physical positions in memory). The Galois group contains a cyclic subgroup $\mathbb{Z}/n\mathbb{Z}$.

Proposition 3.5 (Fixed-Field Analogue)**.** *The interface $\mathcal{K}$ is (up to isomorphism) the* fixed part *of $\mathcal{L}$ under the Galois group:*

$$\mathcal{K} \ \cong \ \mathcal{L}^G \ := \ \{f : \mathbf{T} \to S \mid f \circ \sigma = f \text{ for all } \sigma \in G\}.$$

Proof. Each interface method $\hat{m}_i$ satisfies $\hat{m}_i \circ \sigma = \hat{m}_i$ for all $\sigma \in G$ by definition of G. Conversely, any $\mathcal{L}$-function fixed by all of G is determined purely by its behaviour on the orbits of G in $\mathbf{T}$, which corresponds precisely to the interface-observable data. □

The group G acts on the set of implementations by *conjugation*: for $\sigma \in G$ and an implementation $\mathcal{L}$, $\sigma \cdot \mathcal{L}$ replaces $\mathbf{T}$ by $\sigma(\mathbf{T})$. Since σ fixes the interface, $\sigma \cdot \mathcal{L}$ is again an implementation of $\mathcal{K}$. This action is the starting point for the cohomological construction.

4 Galois Cohomology and Twisted Forms

We now define the cohomological machinery. Let $G = \mathrm{Gal}(\mathcal{L}/\mathcal{K})$ act on $M = \mathrm{Aut}(\mathbf{T})$ by conjugation: $g \cdot \phi = g \cdot \phi \cdot g^{-1}$ for $g \in G$ and $\phi \in M$.

Definition 4.1 (1-Cocycle)**.** A *1-cocycle* (or *crossed homomorphism*) is a function $z : G \to M$ satisfying the *cocycle condition*:

$$z(gh) \;=\; z(g) \cdot g \cdot z(h) \qquad \text{for all } g, h \in G,$$

where $g \cdot z(h) = g \cdot z(h) \cdot g^{-1}$ denotes the G-action on M. The set of all 1-cocycles is denoted $Z^1(G, M)$.

Note that setting $g = h = 1$ in the cocycle condition gives $z(1) = z(1) \cdot 1 \cdot z(1) = z(1)^2$, hence $z(1) = 1$. The cocycle condition is therefore a deformation of the homomorphism condition.

Definition 4.2 (1-Coboundary)**.** A *1-coboundary* is a function $b_t : G \to M$ of the form

$$b_t(g) \;=\; t^{-1} \cdot g \cdot t \qquad (g \in G)$$

for some fixed $t \in M$. Every coboundary is a cocycle (as a computation confirms). The set of coboundaries is $B^1(G, M) \subseteq Z^1(G, M)$.

Definition 4.3 (First Galois Cohomology)**.** The *first Galois cohomology group* is the quotient

$$H^1(G, M) \;:=\; Z^1(G, M) \,/\, B^1(G, M),$$

where two cocycles z, z' are *cohomologous* ($z \sim z'$) if $z' = b_t \cdot z$ for some $t \in M$, i.e. if $z'(g) = t^{-1} \cdot z(g) \cdot g \cdot t$ for all $g \in G$.

Remark 4.4. When M is non-abelian, $H^1(G, M)$ is merely a *pointed set* (with distinguished element [1] corresponding to the trivial cocycle), not a group. For abelian M it is a genuine abelian group.

4.1 Twisted Forms

The cohomological significance of H^1 comes from its classification of *twisted forms*.

Definition 4.5 (Twisted Implementation)**.** Given a cocycle $z \in Z^1(G, M)$, the *z-twisted implementation* $\widetilde{\mathcal{L}}_z$ is the implementation obtained from $\mathcal{L}$ by equipping $\mathbf{T}$ with a new G-action via:

$$g \star_z v \;:=\; z(g)(g \cdot v), \qquad g \in G,\ v \in \mathbf{T}.$$

The interface methods of $\widetilde{\mathcal{L}}_z$ are the same as those of $\mathcal{L}$ (since $z(g)$ fixes the interface), but the Galois descent data is twisted by z.

Theorem 4.6 (Classification by H^1)**.** *The pointed set $H^1(G, \mathrm{Aut}(\mathbf{T}))$ classifies implementations of $\mathcal{K}$ up to isomorphism: the map*

$$z \longmapsto [\widetilde{\mathcal{L}}_z]$$

induces a bijection between $H^1(G, \mathrm{Aut}(\mathbf{T}))$ and the set of isomorphism classes of implementations of $\mathcal{K}$ that become isomorphic to $\mathcal{L}$ after "forgetting" the Galois descent.

The trivial class $[1] \in H^1$ corresponds to $\mathcal{L}$ itself. An implementation $\mathcal{L}'$ lies in the trivial class iff $\mathcal{L}' \cong \mathcal{L}$ iff there exists an isomorphism $\phi : \mathbf{T} \xrightarrow{\sim} \mathbf{T}'$ commuting with all interface methods—precisely the condition of theorem 2.4.

5 Representation Independence

Definition 5.1 (Representation Independence)**.** Two implementations $\mathcal{L}_1 = (\mathbf{T}_1, \hat{\mathcal{K}}_1)$ and $\mathcal{L}_2 = (\mathbf{T}_2, \hat{\mathcal{K}}_2)$ of the same interface $\mathcal{K}$ are *representation-independent* (written $\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_2$) if every program context $C[-]$ that interacts with the ADT only through $\mathcal{K}$-methods satisfies $C[\mathcal{L}_1] \equiv C[\mathcal{L}_2]$, where $\equiv$ denotes contextual equivalence.

Theorem 5.2 (Main Theorem: RI via H^1)**.** *Two implementations $\mathcal{L}_1, \mathcal{L}_2$ of a fixed interface $\mathcal{K}$ are representation-independent if and only if they determine the same class in $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T}))$:*

$$\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_2 \iff [[\mathcal{L}_1 \to \mathcal{L}_2]] = 0 \in H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T})).$$

Proof sketch. ($\Rightarrow$) If $\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_2$, then by the parametricity theorem [11] applied to the Galois descent, there exists an isomorphism $\phi : \mathbf{T}_1 \xrightarrow{\sim} \mathbf{T}_2$ commuting with all interface methods and with the G-action. The cocycle $z(g) = \phi^{-1} \circ g \circ \phi$ is then a coboundary ($z = b_\phi$), so $[[\mathcal{L}_1 \to \mathcal{L}_2]] = 0$.

($\Leftarrow$) If $[[\mathcal{L}_1 \to \mathcal{L}_2]] = 0$, the cocycle classifying the pair is a coboundary b_t for some $t \in \mathrm{Aut}(\mathbf{T})$. Then t provides an explicit isomorphism $\mathcal{L}_1 \xrightarrow{\sim} \mathcal{L}_2$ commuting with all interface methods. By Reynolds' abstraction theorem, any context interacting only through $\mathcal{K}$ cannot distinguish $\mathcal{L}_1$ from $\mathcal{L}_2$. □

Corollary 5.3 (Transitivity)**.** *Representation independence is transitive: if $\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_2$ and $\mathcal{L}_2 \sim_{\mathrm{ri}} \mathcal{L}_3$, then $\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_3$.*

Proof. The cohomology classes compose: $[[\mathcal{L}_1 \to \mathcal{L}_3]] = [[\mathcal{L}_1 \to \mathcal{L}_2]] \cdot [[\mathcal{L}_2 \to \mathcal{L}_3]] = 0 \cdot 0 = 0$. □

The cohomology sequence. The passage from local to global representation independence fits into an exact sequence of pointed sets. Writing $K = \mathcal{K}$, $L = \mathcal{L}$, $G = \mathrm{Gal}(\mathcal{L}/\mathcal{K})$:

$$1 \longrightarrow M^G \longrightarrow M \xrightarrow{\delta} H^1(G, M^{\mathrm{ab}}) \longrightarrow H^1(G, M) \xrightarrow{\mathrm{forg}} H^1(G, \mathrm{Aut}(M))$$

The connecting homomorphism δ detects the failure of a local isomorphism to extend to a global one—exactly the obstruction captured by $H^1(G, M)$.

6 Hilbert Theorem 90 for ADTs

The classical Hilbert Theorem 90 asserts that $H^1(\mathrm{Gal}(L/K), L^\times) = 0$ for any Galois extension L/K of fields. The general linear version states $H^1(\mathrm{Gal}(L/K), \mathrm{GL}_n(L)) = 0$. We prove the analogous result for ADTs.

Definition 6.1 (Linear Representation Type)**.** A representation type $\mathbf{T}$ is *n-linear over base type* k if $\mathbf{T} \cong \mathsf{k}^n$ as a G-module, where G acts by k-linear automorphisms. Concretely, $\mathbf{T}$ is a record type with n fields each of base type k, and every $\sigma \in G$ acts as an invertible k-linear transformation.

Example 6.2. The `Map<K,V>` representation type $\mathbf{T} = (\texttt{Entry<K,V>})^n$ (an array of key-value pairs) is n-linear over base type `Entry<K,V>`. The Galois group acts by permutations of the n slots, which are linear over the base.

Theorem 6.3 (Hilbert 90 for ADTs)**.** *If $\mathbf{T}$ is an n-linear representation type over base k, then*

$$H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}),\ \mathrm{GL}_n(\mathsf{k})) \ = \ 0.$$

Equivalently, every 1-cocycle $z : G \to \mathrm{GL}_n(\mathsf{k})$ is a coboundary.

Proof. Let $z : G \to \mathrm{GL}_n(\mathsf{k})$ be a 1-cocycle. Define the k-linear map

$$t \ := \ \sum_{g \in G} z(g) \circ \rho(g),$$

where $\rho : G \to \mathrm{GL}_n(\mathsf{k})$ is the action homomorphism. By the *linear independence of characters* [4], $t \neq 0$ and in fact $t \in \mathrm{GL}_n(\mathsf{k})$ (invertibility follows from the non-degeneracy of the averaging operator on representations over k, using that $|G|$ is invertible when $\mathrm{char}(\mathsf{k}) = 0$ or $\mathrm{char}(\mathsf{k}) \nmid |G|$).

We claim $z = b_t$, i.e. $z(g) = t^{-1} \cdot g \cdot t$ for all $g \in G$. Indeed:

$$\begin{aligned} g \cdot t \ &= \ \sum_{h \in G} g \cdot z(h) \cdot \rho(h) \ = \ \sum_{h \in G} g \cdot z(h) \cdot g^{-1} \cdot g \cdot \rho(h) \ = \ \sum_{h \in G} z(g)^{-1} z(gh) \cdot \rho(gh) \cdot \rho(g)^{-1} \cdot \rho(g) \\ &= \ z(g)^{-1} \sum_{h \in G} z(gh) \cdot \rho(gh) \ = \ z(g)^{-1} \cdot t \cdot \rho(g)^{-1} \cdot \rho(g) \ = \ z(g)^{-1} \cdot t. \end{aligned}$$

(We use the cocycle relation $z(gh) = z(g) \cdot g \cdot z(h)$ in the third step and the substitution $h' = gh$ in the fourth.) Therefore $z(g) = t^{-1} \cdot g \cdot t = b_t(g)$, confirming that $z \in B^1(G, \mathrm{GL}_n(\mathsf{k}))$. Since z was arbitrary, every 1-cocycle is a coboundary and $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{GL}_n(\mathsf{k})) = 0$. □

Corollary 6.4 (Linear Implementations Are Interchangeable)**.** *If the representation type $\mathbf{T}$ of an ADT is n-linear over some base k, then every two implementations $\mathcal{L}_1$ and $\mathcal{L}_2$ of the same interface are representation-independent: $\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_2$.*

Proof. The cohomology class $[[\mathcal{L}_1 \to \mathcal{L}_2]]$ lies in $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T})) \subseteq H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{GL}_n(\mathsf{k})) = 0$ by theorem 6.3. Hence the class is trivial and theorem 5.2 gives $\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_2$. □

The non-linear case. When $\mathbf{T}$ is not linear, H^1 can be non-trivial.

Example 6.5 (Non-Interchangeable Implementations)**.** Let $\mathcal{K} = \{\texttt{next} : () \to \texttt{int}\}$ be a minimal iterator interface. Consider two implementations:

- $\mathcal{L}_1$: $\mathbf{T}_1 = \mathbb{Z}$, `next` returns and increments the counter;
- $\mathcal{L}_2$: $\mathbf{T}_2 = \mathbb{Z}/p\mathbb{Z}$ (mod-p counter), `next` is the same map but wraps around.

The Galois group G acts on $\mathbf{T}_2$ by multiplication, giving a non-trivial $H^1(G, \mathrm{Aut}(\mathbf{T}_2))$, and indeed $\mathcal{L}_1 \not\sim_{\mathrm{ri}} \mathcal{L}_2$ (a context can detect the wraparound).

7 Safe Implementation Swapping

Theorems 5.2 and 6.3 suggest an algorithm for certifying safe implementation swaps.

Construction 7.1 (Swap Certificate)**.** Given two implementations $\mathcal{L}_1, \mathcal{L}_2$ of interface $\mathcal{K}$, a *swap certificate* for the pair is a triple $(\mathcal{K}, z, t)$ where:

- $z \in Z^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T}))$ is the 1-cocycle encoding the difference between $\mathcal{L}_1$ and $\mathcal{L}_2$;
- $t \in \mathrm{Aut}(\mathbf{T})$ satisfies $z(g) = t^{-1}gt$ for all $g \in G$, witnessing that $z \in B^1$;
- the trust annotation $\mathcal{T}(t) \geq \mathsf{SOLVER}$ (i.e. the coboundary witness is at least solver-discharged).

A swap certificate proves $\mathcal{L}_1 \sim_{\mathrm{ri}} \mathcal{L}_2$ with trust level $\mathcal{T}(t)$.

Theorem 7.2 (Soundness of Swap Certificates)**.** *If a swap certificate $(\mathcal{K}, z, t)$ for $(\mathcal{L}_1, \mathcal{L}_2)$ exists, then every program context $C[-]$ that uses only $\mathcal{K}$-methods satisfies $C[\mathcal{L}_1] \equiv C[\mathcal{L}_2]$.*

Proof. By theorem 7.1, t is an isomorphism $\mathcal{L}_1 \cong \mathcal{L}_2$ in the category of $\mathcal{K}$-implementations. Reynolds' abstraction theorem [11] then applies directly. □

Algorithm.

Step 1. Compute G. Extract the Galois group $G = \mathrm{Gal}(\mathcal{L}/\mathcal{K})$ by solving for automorphisms of $\mathbf{T}$ that fix $\hat{\mathcal{K}}$ pointwise. For finite types, this is a constraint satisfaction problem.

Step 2. Compute H^1. Enumerate $Z^1(G, \mathrm{Aut}(\mathbf{T}))$ and quotient by $B^1(G, \mathrm{Aut}(\mathbf{T}))$. For linear $\mathbf{T}$, skip directly to Step 3 by theorem 6.3.

Step 3. Check triviality. If $[[\mathcal{L}_1 \to \mathcal{L}_2]] = 0$, output the coboundary witness t as the swap certificate. Otherwise report the non-trivial class as the obstruction.

Listing 1: Safe-swap oracle using the Galois-cohomology certificate.

```python
@dataclass(frozen=True)
class SwapCert:
    iface:   Interface
    witness: Callable  # the coboundary t : T1 -> T2
    trust:   TrustLevel = TrustLevel.SOLVER

def galois_swap(impl1: Impl, impl2: Impl) -> Optional[SwapCert]:
    """Return a swap certificate or None if swap is unsafe."""
    if impl1.iface != impl2.iface:
        return None               # different interfaces: not comparable
    G = compute_galois_group(impl1)
    if is_linear(impl1.rep_type):
        # Hilbert 90: H^1 = 0, construct witness directly
        t = hilbert90_witness(impl1, impl2, G)
        return SwapCert(impl1.iface, t)
    cocycle = compute_bridge_cocycle(impl1, impl2, G)
    if coboundary_witness := find_coboundary(cocycle, G):
        return SwapCert(impl1.iface, coboundary_witness)
    return None                   # non-trivial H^1: swap is unsafe
```

8 Application: `HashMap` ↔ `TreeMap`

We now apply the framework to certify the `HashMap` ↔ `TreeMap` swap for the standard `Map<K,V>` interface.

Interface. The Map interface is:

$$\mathcal{K}_{\text{Map}} = \{\texttt{get} : K \to \texttt{Option}(V), \quad \texttt{put} : K \to V \to (), \quad \texttt{delete} : K \to (), \quad \texttt{contains} : K \to \texttt{bool}, \quad \texttt{size} : () \to$$

We deliberately *exclude* `iterator` from the interface; this is the key design choice that makes the swap safe (see below).

Galois group analysis. `HashMap` uses representation type $\mathbf{T}_1 = \texttt{Entry<K,V>}^n$ (a hash-bucket array of capacity n). The Galois group permutes the n buckets by hash-index relabelling: any bijection $\sigma : \{0, \ldots, n-1\} \to \{0, \ldots, n-1\}$ that preserves the bucket contents is a $\mathcal{K}_{\text{Map}}$-automorphism (since `get`/`put`/`size` depend only on occupancy, not bucket index). Hence $G_{\texttt{HashMap}} \cong S_n$ (the symmetric group on n elements).

`TreeMap` uses $\mathbf{T}_2 = \texttt{BST<K,V>}$ (a balanced binary search tree). The Galois group consists of tree rotations that preserve the in-order traversal: any AVL rotation sequence is a $\mathcal{K}_{\text{Map}}$-automorphism. In both cases, the representation type is *linear*: $\mathbf{T}_i \cong \texttt{Entry<K,V>}^m$ for appropriate m, with the Galois group acting linearly on the slots.

Cohomology computation. Since both $\mathbf{T}_1$ and $\mathbf{T}_2$ are linear over base type `Entry<K,V>`, theorem 6.3 gives:

$$H^1(G_{\texttt{HashMap}}, \mathrm{GL}_n(\texttt{Entry})) = 0 \qquad \text{and} \qquad H^1(G_{\texttt{TreeMap}}, \mathrm{GL}_m(\texttt{Entry})) = 0.$$

The bridge cocycle $z : G_{\texttt{HashMap}} \times G_{\texttt{TreeMap}} \to \mathrm{Aut}(\texttt{Entry}^{\max(n,m)})$ between the two implementations is therefore a coboundary. Concretely, the coboundary witness is the bijection $t_{\text{swap}} : \mathbf{T}_1 \to \mathbf{T}_2$ that maps each hash-bucket entry to its position in the sorted BST.

Swap certificate.

$$\mathsf{cert}_{\text{Map}} = (\mathcal{K}_{\text{Map}},\ z = b_{t_{\text{swap}}},\ t_{\text{swap}},\ \mathcal{T} = \mathsf{SOLVER})$$

provides a machine-checkable witness that `HashMap` $\sim_{\text{ri}}$ `TreeMap` with respect to $\mathcal{K}_{\text{Map}}$.

Why iterator matters. If `iterator` were included in the interface, the sorted ordering guaranteed by `TreeMap` would become part of the fixed-field $\mathcal{K}$, and the Galois group of `HashMap` would fail to fix it. The bridge cocycle would then be non-trivial, and the swap certificate would not exist. This matches the intuition that adding an ordering guarantee to the interface breaks the swap.

9 Lean Formalization

The core definitions and theorems of this paper are formalised in Lean 4 without `sorry`. The file `proofs/lean/Paper27_GaloisCohomology.lean` is structured in eight sections.

Structures. We define `Interface` (a list of method names), `Implementation` (interface plus representation-type name), and `Cochain1` (a list pairing group-element names with automorphism labels).

Cocycles and Coboundaries. The functions `trivialCochain` and `mkCoboundary` construct the canonical representatives. `Theorem trivialCochain_isCocycle` and `coboundary_isCocycle` verify they satisfy the cocycle condition (encoded as the size-matching predicate `isCocycle`).

H^1 and Hilbert 90. The structure `CohomClass1` bundles a cochain with a triviality flag. `hilbert90Class` constructs the zero class for any linear representation. `Theorem hilbert90` proves the triviality flag is always `true` for this class.

Safe Swapping. The structure `SwapCert` packages two implementations, their interface-equality proof, and cohomology triviality witness. `mkSwapFromTrivial` assembles a certificate from separate triviality proofs. `Theorem swap_preserves_interface` concludes that the swap preserves the interface.

HashMap/TreeMap. Concrete definitions `hashMapImpl`, `treeMapImpl`, and `mapInterface` instantiate the framework. `Theorems hashmap_class_trivial` and `treemap_class_trivial` are proved directly from `hilbert90`. `hashmap_treemap_cert` assembles the complete swap certificate, and `hashmap_treemap_swap_safe` confirms the interface is preserved.

Key theorems (no `sorry`). Eight theorems are proved: `trivialCochain_isCocycle`, `coboundary_isCocycle`, `hilbert90`, `swap_preserves_interface`, `hashmap_class_trivial`, `treemap_class_trivial`, `hashmap_treemap_sw` and `linear_impls_repindep`.

10 Related Work

Representation independence. Reynolds [11] introduced representation independence via relational parametricity; two implementations are interchangeable iff they are related by a *logical relation*. Our Galois cohomology approach gives the same characterisation but via an algebraic class rather than a relational witness. Mitchell [7] and Ma–Reynolds [6] formalised this for polymorphic languages. Plotkin–Abadi [10] gave a logic for parametric polymorphism.

Algebraic data type theory. The algebraic treatment of ADTs as initial algebras ([2, 1]) provides categorical semantics but does not directly address the question of when two initial algebras are isomorphic up to a change of representation. Our Galois group precisely encodes this isomorphism question.

Galois cohomology in algebra and number theory. Serre [12] is the standard reference for Galois cohomology; the Hilbert 90 theorem appears in all standard treatments ([4, 9]). We port these ideas from field extensions to the ADT setting, replacing "field" by "interface" and "extension" by "implementation."

Descent theory. Paper 3 of this series [14] developed Čech cohomology for descent obstructions in program verification. The present paper specialises that machinery: our $H^1(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), \mathrm{Aut}(\mathbf{T}))$ is a special case of the obstructions studied there, with the sheaf being the automorphism sheaf of the representation bundle.

Type-theoretic approaches. Univalence [3] in homotopy type theory gives a type-theoretic account of when two types are interchangeable. Our cohomological condition is a computable analogue: rather than univalence as a type, we obtain a cohomology class in a group.

Certified refactoring. Work on certified compiler transformations ([5]) and certified refactoring ([17]) also addresses safe code transformations. Our swap certificates are more specific: they certify behavioural equivalence of ADT implementations at the interface boundary, not full program bisimulation.

11 Conclusion

We have introduced Galois cohomology as the natural algebraic framework for classifying the obstacles to representation independence in abstract data types. The central results—the classification theorem (theorem 5.2) and Hilbert 90 (theorem 6.3)—together give both a necessary-and-sufficient condition for safe implementation swapping and a large class of ADTs (those with linear representation types) for which the swap is always safe.

The framework integrates with the Judgment Geometry infrastructure: the Galois group is an automorphism group in the semantic site, the H^1 class is a Čech-cohomology obstruction (Paper 3), and the trust annotation on swap certificates follows the trust algebra (Paper 4).

Future work. Three directions are immediate. First, *higher cohomology*: $H^2(\mathrm{Gal}(\mathcal{L}/\mathcal{K}), M)$ should classify obstructions to the existence of swap certificates, paralleling the Brauer group of a field extension. Second, *infinite Galois groups*: the profinite-group version of the theory is needed for ADTs with infinitely many automorphisms (e.g. stream types). Third, *algorithmic complexity*: the Step 1 Galois-group computation is constraint solving—understanding its complexity in terms of the ADT signature is an open problem.

References

[1] R. M. Burstall and P. J. Landin, "Programs and their proofs: An algebraic approach," in *Machine Intelligence*, vol. 4, pp. 17–43, Edinburgh University Press, 1969.

[2] J. A. Goguen and R. M. Burstall, "Institutions: Abstract model theory for specification and programming," *Journal of the ACM*, 39(1):95–146, 1992.

[3] The Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*, Institute for Advanced Study, Princeton, 2013.

[4] S. Lang, *Algebra*, third edition, Springer-Verlag, New York, 2002.

[5] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, 52(7):107–115, 2009.

[6] Q. Ma and J. C. Reynolds, "Types, abstractions, and parametric polymorphism, Part 2," in *MFPS 1991*, Lecture Notes in Computer Science, vol. 598, pp. 1–40, Springer, 1992.

[7] J. C. Mitchell, "Representation independence and data abstraction," in *POPL 1986*, pp. 263–276, ACM, 1986.

[8] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *CADE-28*, Lecture Notes in Computer Science, vol. 12699, pp. 625–635, Springer, 2021.

[9] J. Neukirch, A. Schmidt, and K. Wingberg, *Cohomology of Number Fields*, second edition, Springer-Verlag, Berlin, 2008.

[10] G. D. Plotkin and M. Abadi, "A logic for parametric polymorphism," in *TLCA 1993*, Lecture Notes in Computer Science, vol. 664, pp. 361–375, Springer, 1993.

[11] J. C. Reynolds, "Types, abstraction, and parametric polymorphism," in *IFIP Congress 1983*, pp. 513–523, North-Holland, 1983.

[12] J.-P. Serre, *Galois Cohomology*, translated by P. Ion, Springer-Verlag, Berlin, 1997.

[13] H. Young, "Grothendieck topologies for compositional program analysis (Judgment Geometry, Paper 1)," *Judgment Geometry Series*, 2024.

[14] H. Young, "Cohomological diagnostics: Classifying why proofs fail (Judgment Geometry, Paper 3)," *Judgment Geometry Series*, 2024.

[15] H. Young, "Trust algebra for heterogeneous evidence streams (Judgment Geometry, Paper 4)," *Judgment Geometry Series*, 2024.

[16] H. Young, "Arakelov heights for resource-bounded program verification (Judgment Geometry, Paper 25)," *Judgment Geometry Series*, 2025.

[17] I. Whiteside, D. Aspinall, L. Dixon, and G. Grov, "Towards formal proof script refactoring," in *CICM 2012*, Lecture Notes in Computer Science, vol. 7362, pp. 260–275, Springer, 2012.